

ARVENA v1.0

TECHNICAL ARCHITECTURE & DATA BLUEPRINT

Status: BUILD-READY

1. ARCHITECTURE GOALS

The architecture must support:

- multi-tenancy;
 - mobile-first;
 - eventual global scale;
 - modular industries;
 - reliable background work;
 - low-cost early deployment;
 - strong tenant isolation;
 - independent external integrations.
-

2. SYSTEM LAYERS

Client

Responsive Web/PWA.

Responsibilities:

- UI;
- local form state;
- optimistic UX;
- offline-tolerant worker actions;
- upload preparation.

Application Layer

Server routes/actions.

Responsibilities:

- authorization;

- business rules;
- transaction orchestration;
- secure integrations.

Database

PostgreSQL.

Responsibilities:

- persistence;
- constraints;
- RLS;
- transactional integrity.

Queue Workers

Responsibilities:

- reminders;
- automation;
- imports;
- outbound integrations;
- retries.

Storage

Private object storage.

External Integrations

WhatsApp, maps, payments later.

3. DOMAIN BOUNDARIES

Identity

organizations
members
roles
permissions

CRM

customers
leads
follow_ups
notes

Sales

quotes
service_catalog

Work

jobs
visits
assets
checklists
media

Scheduling

visit assignments
availability
leave
recurring rules

Billing

invoices
payments

Communication

integrations
external events
notifications

Automation

rules
runs
queue jobs

Insights

activities
audit logs
report queries

4. ID STRATEGY

Primary keys: UUID.

Never expose sequential internal IDs as authorization mechanism.

Human-readable numbers separate.

Examples:

JOB-2026-000001 INV-2026-000001 QTE-2026-000001

Number sequence scoped by organization.

5. CORE COMMON COLUMNS

Tenant tables generally include:

id uuid

organization_id uuid

created_at timestamptz

updated_at timestamptz

created_by uuid nullable

updated_by uuid nullable

archived_at timestamptz nullable

Critical mutable records additionally:

version integer default 1

6. ORGANIZATIONS

organizations

- id
- name
- slug
- country_code
- currency_code
- timezone
- locale
- industry_template_id
- subscription_plan
- status
- logo_media_id
- created_at
- updated_at

Constraints:

currency uses ISO-style code.

Timezone uses IANA timezone identifier.

7. ORGANIZATION MEMBERS

organization_members

- id
- organization_id
- user_id
- role_id
- branch_id nullable
- status
- invited_at
- joined_at
- disabled_at

Unique:

organization_id + user_id

8. ROLES

roles

- id
- organization_id nullable
- name
- system_key nullable
- is_system_role
- created_at

System keys:

OWNER
ADMIN
SALES
WORKER
SUPERVISOR
FINANCE

9. ROLE PERMISSIONS

role_permissions

- role_id
- permission_key
- allowed boolean

Examples:

customer.view

customer.create

job.assign

invoice.issue

settings.manage

10. CUSTOMERS

customers

- id
- organization_id
- type
- name
- normalized_phone
- whatsapp_phone
- alternate_phone
- email
- source
- assigned_sales_id nullable
- status
- metadata jsonb
- created_at
- updated_at
- archived_at

Index: `organization_id, normalized_phone`

Suggested uniqueness:

unique non-null normalized phone per organization where business rule applies.

If shared phone numbers are expected in future, duplicate override must be explicitly represented rather than silently breaking uniqueness.

11. CUSTOMER ADDRESSES

customer_addresses

- id
- organization_id
- customer_id
- label
- line1
- line2
- city
- region
- postal_code
- country_code
- latitude decimal

- longitude decimal
 - is_primary
-

12. CUSTOMER TAGS

tags

- id
- organization_id
- name

customer_tags

- customer_id
- tag_id

Unique pair.

13. LEADS

leads

- id
 - organization_id
 - customer_id
 - title
 - description
 - service_catalog_id nullable
 - status
 - temperature
 - source
 - assigned_sales_id
 - estimated_value_minor nullable
 - currency_code
 - next_follow_up_at nullable
 - won_at nullable
 - lost_at nullable
 - lost_reason nullable
 - created_at
 - updated_at
 - archived_at
 - version
-

14. LEAD STATUS HISTORY

lead_status_history

- id
- organization_id
- lead_id
- from_status
- to_status
- changed_by
- changed_at

Append-only.

15. FOLLOW UPS

follow_ups

- id
- organization_id
- customer_id
- lead_id nullable
- quote_id nullable
- job_id nullable
- invoice_id nullable
- assigned_to
- reason
- due_at
- status
- completed_at
- snoozed_from nullable
- notes nullable
- created_at

Statuses: PENDING

DONE

CANCELLED

16. SERVICE CATALOG

service_catalog

- id
 - organization_id
 - item_type
 - name
 - description
 - unit
 - unit_price_minor
 - currency_code
 - tax_behavior
 - active
-

17. QUOTES

quotes

- id
- organization_id
- quote_number
- customer_id
- lead_id nullable
- status
- currency_code
- subtotal_minor
- discount_minor
- tax_minor
- total_minor
- expires_at
- issued_at
- accepted_at
- customer_snapshot jsonb
- terms_snapshot text
- version
- created_at

Unique: organization + quote_number

18. QUOTE ITEMS

quote_items

- id
 - organization_id
 - quote_id
 - service_catalog_id nullable
 - name_snapshot
 - description_snapshot
 - unit_snapshot
 - quantity decimal
 - unit_price_minor
 - discount_minor
 - tax_minor
 - total_minor
 - sort_order
-

19. JOBS

jobs

- id
- organization_id
- job_number
- customer_id
- lead_id nullable
- quote_id nullable
- title
- description
- status
- priority
- branch_id nullable
- primary_address_id nullable
- currency_code
- estimated_amount_minor nullable
- scheduled_start nullable
- scheduled_end nullable
- completed_at nullable
- cancelled_at nullable
- cancellation_reason nullable
- version
- created_at
- updated_at

- archived_at

Unique: organization + job_number

20. JOB ITEMS

job_items

- id
 - organization_id
 - job_id
 - service_catalog_id nullable
 - name_snapshot
 - quantity
 - unit_price_minor nullable
 - notes
-

21. VISITS

visits

- id
- organization_id
- job_id
- visit_type
- status
- scheduled_start
- scheduled_end
- actual_departed_at
- actual_arrived_at
- actual_started_at
- actual_completed_at
- address_snapshot jsonb
- timezone
- completion_notes
- version
- created_at
- updated_at

Index: organization + scheduled_start

Index: job_id

22. VISIT ASSIGNMENTS

visit_assignments

- id
- organization_id
- visit_id
- member_id
- assignment_role
- assigned_at
- removed_at nullable

Unique active assignment: visit + member.

23. WORKER AVAILABILITY

member_availability

- id
 - organization_id
 - member_id
 - weekday
 - start_local_time
 - end_local_time
 - timezone
-

24. LEAVE

member_leave

- id
 - organization_id
 - member_id
 - starts_at
 - ends_at
 - reason
 - status
-

25. CHECKLIST TEMPLATES

checklist_templates

- id
- organization_id nullable
- industry_template_id nullable
- name
- active

checklist_template_items

- id
 - checklist_template_id
 - input_type
 - label
 - required
 - config jsonb
 - sort_order
-

26. JOB CHECKLISTS

job_checklists

- id
- organization_id
- job_id
- visit_id nullable
- template_snapshot jsonb
- status

checklist_responses

- id
 - organization_id
 - job_checklist_id
 - template_item_key
 - value jsonb
 - completed_by
 - completed_at
-

27. ASSETS

assets

- id
- organization_id
- customer_id
- asset_type
- name
- brand
- model
- serial_number
- installed_at
- warranty_until
- status
- metadata jsonb
- archived_at

Index: organization + serial number where useful.

28. ASSET SERVICE HISTORY

May be derived from Jobs, but explicit relationship table:

job_assets

- organization_id
- job_id
- asset_id

Unique pair.

29. MEDIA

media

- id
- organization_id
- customer_id nullable
- job_id nullable
- visit_id nullable
- asset_id nullable

- category
- storage_key
- mime_type
- size_bytes
- upload_status
- uploaded_by
- created_at

Storage key must include organization identifier.

30. NOTES

notes

- id
- organization_id
- entity_type
- entity_id
- body
- visibility
- author_id
- created_at
- updated_at
- archived_at

Polymorphic access must still be tenant checked.

31. INVOICES

invoices

- id
- organization_id
- invoice_number
- customer_id
- job_id nullable
- quote_id nullable
- status
- currency_code
- subtotal_minor
- discount_minor
- tax_minor
- total_minor

- paid_minor
- outstanding_minor
- issued_at
- due_at
- paid_at nullable
- customer_snapshot jsonb
- terms_snapshot text
- version
- created_at
- updated_at

Unique: organization + invoice_number

32. INVOICE ITEMS

invoice_items

- id
 - organization_id
 - invoice_id
 - name_snapshot
 - description_snapshot
 - quantity
 - unit_snapshot
 - unit_price_minor
 - discount_minor
 - tax_minor
 - total_minor
 - sort_order
-

33. PAYMENTS

payments

- id
- organization_id
- invoice_id
- amount_minor
- currency_code
- method
- reference
- paid_at
- received_by

- notes
- status
- created_at

Payment changes must recalculate invoice totals transactionally.

34. RECURRING RULES

recurring_rules

- id
- organization_id
- customer_id
- service_catalog_id nullable
- asset_id nullable
- recurrence_type
- interval_value
- weekday_config jsonb
- local_time
- timezone
- starts_on
- ends_on nullable
- next_occurrence_at
- active
- template_data jsonb
- created_at
- updated_at

35. RECURRING OCCURRENCES

recurring_occurrences

- id
- organization_id
- recurring_rule_id
- occurrence_at
- generated_job_id nullable
- status

Unique: recurring_rule + occurrence timestamp.

Prevents duplicate generation.

36. ACTIVITIES

activities

Append-only operational timeline.

- id
 - organization_id
 - entity_type
 - entity_id
 - actor_id nullable
 - event_type
 - summary
 - metadata jsonb
 - occurred_at
-

37. AUDIT LOGS

audit_logs

Append-only.

- id
- organization_id
- actor_id nullable
- action
- entity_type
- entity_id
- before_json
- after_json
- ip_address nullable
- user_agent nullable
- occurred_at

Sensitive fields may require redaction.

38. INTEGRATIONS

integrations

- id

- organization_id
- provider
- status
- external_account_id
- display_name
- secret_reference
- last_success_at
- last_error_at
- last_error_code
- metadata
- created_at
- updated_at

Secrets should not be directly stored in general readable metadata.

39. EXTERNAL EVENTS

external_events

- id
- organization_id nullable
- provider
- external_event_id
- event_type
- payload_hash
- processing_status
- received_at
- processed_at nullable
- error_code nullable

Unique: provider + external_event_id

Core idempotency mechanism.

40. AUTOMATION RULES

automation_rules

- id
- organization_id
- name
- trigger_type
- conditions jsonb

- actions jsonb
 - active
 - created_by
 - created_at
 - updated_at
-

41. AUTOMATION RUNS

automation_runs

- id
- organization_id
- automation_rule_id
- trigger_event_id
- status
- scheduled_for nullable
- started_at nullable
- finished_at nullable
- retry_count
- last_error
- idempotency_key

Unique where appropriate: automation rule + triggering event.

42. NOTIFICATIONS

notifications

- id
 - organization_id
 - recipient_member_id
 - type
 - title
 - body
 - read_at nullable
 - entity_type nullable
 - entity_id nullable
 - created_at
-

43. SUBSCRIPTION ENTITLEMENTS

subscription_entitlements

Could be config-backed.

- plan_key
- feature_key
- value_json

No duplicated plan logic across components.

44. USAGE COUNTERS

usage_counters

- organization_id
- period_start
- metric_key
- usage_value

Examples: jobs_created storage_bytes automation_runs messages_sent

45. CUSTOM FIELD DEFINITIONS

custom_field_definitions

- id
- organization_id
- entity_type
- key
- label
- field_type
- config
- required
- active

custom_field_values

- id
- organization_id
- definition_id

- entity_id
 - value jsonb
-

46. BRANCHES

branches

- id
 - organization_id
 - name
 - country_code
 - timezone
 - address_json
 - active
-

47. IDENTITY SECURITY

Client receives only public/session-safe credentials.

Privileged service credentials never reach browser.

Every server mutation:

- authenticates;
 - resolves organization context;
 - checks permission;
 - validates payload;
 - performs transaction.
-

48. RLS PRINCIPLE

All exposed tenant data: `organization_id` must equal a membership organization the user can access.

Worker restrictions additionally require assignment relation.

Never rely only on frontend hiding.

49. DATABASE TRANSACTIONS

Required for:

- invoice + items creation;
 - payment + invoice balance update;
 - job conversion;
 - quote acceptance + job creation;
 - recurring generation;
 - membership role changes.
-

50. OPTIMISTIC CONCURRENCY

Critical mutable tables:

- jobs;
- visits;
- quotes;
- invoices;
- automation rules.

Update where: `id = ? AND version = current_version`

Then increment version.

Zero rows updated: conflict.

51. EVENT CATALOG

Internal event naming:

`customer.created`

`customer.updated`

`lead.created`

`lead.status_changed`

`lead.won`

quote.issued

quote.accepted

job.created

job.confirmed

job.scheduled

job.started

job.completed

job.cancelled

visit.scheduled

visit.departed

visit.arrived

visit.started

visit.completed

invoice.issued

invoice.overdue

invoice.paid

payment.recorded

followup.created

followup.completed

asset.created

recurring.occurrence_due

integration.connected

integration.failed

52. EVENT PAYLOAD MINIMUM

Each event includes:

- event_id;
 - organization_id;
 - event_type;
 - entity_type;
 - entity_id;
 - actor_id nullable;
 - occurred_at;
 - payload_version;
 - relevant snapshot/metadata.
-

53. EVENT DELIVERY RULES

Internal business transaction commits first.

Then event is persisted/outboxed.

Background handlers consume reliably.

Do not call critical external APIs before business transaction commits.

54. OUTBOX PATTERN

Recommended for critical event durability.

event_outbox

- id
- organization_id
- event_type
- payload
- status
- created_at
- published_at

Business transaction writes record + outbox event atomically.

Worker publishes/processes later.

55. AUTOMATION EXECUTION

Flow:

Trigger event

- load active matching rules
 - evaluate conditions
 - create automation run
 - schedule action
 - re-check state at execution
 - perform action
 - record result
-

56. RETRY POLICY

External integrations:

Example defaults:

- retry 1: short delay
- retry 2: exponential delay
- retry 3+
- final failure state

Exact delay configurable.

Never retry validation errors endlessly.

57. DEAD LETTER

Failed background job after retry threshold: status = FAILED_FINAL

Expose to admin/integration health where relevant.

58. WEBHOOK FLOW

1. Receive request.
2. Verify signature.
3. Extract external ID.
4. Insert external_event.
5. Duplicate? stop successfully.
6. Queue processing.
7. Respond quickly.
8. Process asynchronously.

Avoid slow webhook responses.

59. WHATSAPP INBOUND PROCESSING

Normalize sender phone.

Find customer in organization.

If no customer: create customer.

Apply intake rule.

Create/link lead/request.

Create activity.

Do not import unrelated historical contacts.

60. FILE UPLOAD FLOW

Client:

- validate type;
- validate size;
- compress image where configured.

Create media record: UPLOADING.

Upload to private storage.

Finalize: UPLOADED.

Failure: FAILED.

Worker may retry without creating duplicate logical media record.

61. OFFLINE WORKER ACTIONS

Local device queue contains:

- client_action_id;
- entity;
- action;
- local_timestamp;
- payload.

Server uses client_action_id as idempotency key.

On reconnect: replay safely.

62. TIME

Store timestamps in UTC.

Store recurrence timezone explicitly.

User-facing display: organization/user timezone.

Never store scheduled recurring business time only as UTC without timezone context.

63. MONEY

Minor-unit integer when currency supports it.

Store: amount + currency.

Never infer currency solely from organization when historical document already exists.

64. NUMBER GENERATION

Human readable document numbers allocated server-side transactionally.

Avoid client-generated sequences.

65. SEARCH

Initial: Postgres search/indexed normalized columns.

Normalize searchable: phone; names; document numbers.

Future dedicated search engine only if necessary.

66. PAGINATION

Prefer cursor/keyset pagination for large operational lists.

Avoid loading thousands of rows.

67. REPORTING

Reports should query prepared aggregates where needed at scale.

Early stage: indexed SQL is acceptable.

Do not prematurely add separate analytics warehouse.

68. CACHE

Use cache only for:

- stable templates;
- plan entitlements;
- low-risk configuration.

Operational state should favor correctness over stale cache.

69. SECURITY LOGGING

Log:

- authentication anomalies;
- denied privileged actions;
- integration errors;
- role changes;
- destructive operations.

Never log secrets.

70. BACKUP

Database: scheduled production backups.

Storage: separate object backup/export strategy.

Configuration: document restore procedure.

Periodic restore test mandatory.

71. ENVIRONMENTS

Minimum:

Development

Staging

Production

Production data must not be casually copied into development.

72. MIGRATION RULES

Database migrations:

- versioned;
- code reviewed;

- reversible where practical;
- destructive migration requires explicit plan.

Never edit historical production migration files after deployment.

73. FEATURE FLAGS

Use for risky rollout:

- WhatsApp connector;
- new automation;
- advanced reports.

Flags not used as permanent architecture replacement.

74. ERROR CODE FORMAT

ARV-`{DOMAIN}`-`{NUMBER}`

Examples:

ARV-JOB-1001

ARV-WA-2003

ARV-BILL-3004

User-facing error stays friendly.

Internal error code allows support diagnostics.

75. MINIMUM OBSERVABILITY

Track:

API errors
queue failures
webhook failure rate
storage failures
automation run failures
DB latency

page/API latency
auth failures

76. PERFORMANCE TARGET

Initial target:

common list API p95 < 800ms under normal expected load.

Common mutation p95 < 1s excluding external provider processing.

UI acknowledges click immediately.

77. RATE LIMITS

Required for:

login attempts; public webhooks; future public forms; bulk imports; message sends; API tokens.

78. API VERSIONING

External/public API later: `/api/v1/...`

Internal APIs may evolve faster.

External webhook contracts versioned.

79. INDUSTRY TEMPLATE DATA

Store templates as configuration:

- terminology;
- statuses;
- checklist definitions;
- suggested fields;
- modules.

Never fork schemas by industry.

80. TECHNICAL LOCK

The following are mandatory:

- one multi-tenant core;
- Job and Visit separate;
- server-authoritative business rules;
- RLS on tenant tables;
- idempotent external/background processing;
- immutable invoice snapshots;
- explicit timezone handling;
- exact money handling;
- soft delete;
- durable event/queue handling;
- no dependency on WhatsApp availability.

These decisions may not be bypassed for implementation convenience.